

Agentic AI

Week 4: LangGraph — Building Agentic AI Workflows

A Comprehensive Study Guide

Topics Covered

- The LangChain Ecosystem
- Introducing LangGraph — Low-Level Orchestration Framework
- LangGraph's Components: Studio and Platform
- Key Terminologies: Graphs, State, Nodes, and Edges
- Building Your First Graph: A Five-Step Process
- Understanding State and Reducers
- Super-Steps and Graph Interactions
- Project Goals: Structured Outputs, Multi-Agent Delegation, Playwright

SECTION 1 — THEORY

1. The LangChain Ecosystem

Before introducing LangGraph specifically, it is essential to understand the broader ecosystem within which it operates. LangChain is the foundational framework upon which LangGraph is built, and the two are deeply complementary.

1.1 What is LangChain?

LangChain is an open-source framework specifically designed for building context-aware, reasoning applications powered by Large Language Models (LLMs). Unlike simple API wrappers, LangChain provides a structured architectural approach for integrating LLMs with external data sources, APIs, and computational tools — enabling the construction of sophisticated AI workflows that go far beyond simple prompt-response interactions.

In the context of enterprise and production AI development, LangChain enables organisations to:

- Build applications that reason over proprietary company data and internal knowledge bases.
- Orchestrate complex, multi-step workflows involving multiple AI calls, data lookups, and tool invocations.
- Develop agentic platforms in which AI models autonomously plan, decide, and execute sequences of actions to accomplish a stated goal.

1.2 The LangChain Ecosystem Components

Term	Definition
LangChain	The core orchestration framework. Provides chains, agents, memory, and integrations with hundreds of external services and data sources.
LangGraph	A low-level graph-based orchestration layer built on top of LangChain. Designed specifically for building stateful, cyclical, multi-agent workflows. The central focus of this week's study.
LangSmith	A monitoring, tracing, and evaluation platform for LLM applications. Integrates with LangGraph to provide full observability into agent behaviour, token usage, latency, and errors in production.

Key Architectural Insight

LangChain provides the building blocks (LLM wrappers, tool integrations, prompt templates).
 LangGraph provides the orchestration layer (stateful graphs, agent control flow, feedback loops).
 LangSmith provides the observability layer (monitoring, debugging, evaluation).

Together, these three form a complete, production-grade Agentic AI development platform.

2. Introducing LangGraph: The Low-Level Orchestration Framework

LangGraph is an open-source AI agent framework developed by the LangChain team. While LangChain handles individual components and simple sequential chains, LangGraph was purpose-built to address a fundamentally different challenge: constructing controllable, stateful, multi-agent workflows that may involve cycles, conditionals, and iterative feedback loops — the hallmarks of truly agentic behaviour.

2.1 What Problems Does LangGraph Solve?

Traditional LLM pipelines are predominantly linear: input flows through a sequence of steps and produces an output. Real-world agentic applications, however, rarely follow a straight line. An agent may need to:

- Call a tool, inspect the result, and decide whether to call it again with a refined query.
- Delegate a sub-task to a specialised agent and await its completion before proceeding.
- Maintain context and memory across multiple turns of interaction.
- Branch into parallel execution paths and merge the results.

LangGraph solves these challenges by representing agent workflows as directed graphs — a mathematical model that naturally accommodates cycles, branches, parallelism, and conditional routing. This gives developers fine-grained control over exactly how an agent reasons and acts.

2.2 Core Characteristics of LangGraph

Term	Definition
Low-Level Framework	Operates closer to the underlying execution model than high-level abstractions. Gives developers explicit control over state transitions, execution order, and agent behaviour.
Stateful Execution	Every step in the workflow operates on a shared, mutable state object. Nodes read from and write to this state, enabling complex multi-step reasoning with memory.
Cyclical Workflows	Unlike simple chains (which are Directed Acyclic Graphs, or DAGs), LangGraph supports cycles — allowing agents to loop, retry, and self-correct iteratively.
Controllable Agents	Developers can define exactly which conditions trigger transitions between nodes, preventing agents from going 'off-script' and ensuring predictable, auditable behaviour.

Abstraction Layer	Acts as an abstraction layer on top of LangChain components, organising complex workflows with feedback loops in a structured, maintainable manner.
--------------------------	---

3. LangGraph's Components

LangGraph provides a comprehensive suite of tools spanning the full development lifecycle — from initial design through production deployment and monitoring.

3.1 LangGraph Studio

LangGraph Studio is a visual builder for designing and prototyping AI agent workflows. It provides a graphical interface in which developers can construct agent graphs by arranging and connecting nodes and edges visually, without writing boilerplate orchestration code. Key capabilities include:

- Visual drag-and-drop graph construction for rapid prototyping.
- Real-time graph state inspection during test runs.
- Interactive debugging: step through agent execution node by node, inspect state at each step.
- Integration with LangSmith for tracing and performance profiling.

Practical Note on LangGraph Studio

LangGraph Studio is particularly valuable during the design phase of an agentic application. Visualising the graph structure makes it significantly easier to reason about complex agent interactions, identify potential deadlocks or infinite loops, and communicate the workflow to non-technical stakeholders.

3.2 LangGraph Platform

LangGraph Platform is the production infrastructure layer for deploying and operating LangGraph agents at scale. It addresses the operational challenges that arise when moving from a prototype to a production system serving real users. It provides:

- Managed infrastructure for running LangGraph agents reliably at scale.
- REST API endpoints for invoking agents from external applications.
- Thread management for handling concurrent user sessions.
- Persistence for checkpointing and resuming long-running agent workflows.
- Built-in integration with LangSmith for production monitoring.

4. Key Terminologies in LangGraph

A precise understanding of LangGraph's core terminology is prerequisite to building effective agent workflows. The following four concepts form the foundational vocabulary of the framework.

4.1 Graphs

In LangGraph, an agent workflow is represented as a directed graph — a mathematical structure composed of nodes (vertices) and edges (connections between vertices). Representing workflows as graphs provides several critical advantages: it naturally models complex decision-making with branches and loops, it is visually inspectable, and it provides a formal basis for reasoning about execution order and state transitions.

A LangGraph graph is specifically a state machine: its execution is driven entirely by the current value of the shared state. The graph begins at a designated start node, transitions between nodes according to the logic defined in its edges, and terminates at a designated end node.

4.2 State

The State is the most fundamental concept in LangGraph. It is a structured data object — typically defined as a Python TypedDict or Pydantic BaseModel — that represents the complete, current snapshot of the agent workflow's information at any point in time.

Every node in the graph receives the current state as its input, performs some computation, and returns an updated state as its output. The state persists and evolves throughout the lifetime of a single graph invocation. Think of it as the agent's working memory — everything the agent knows and has computed so far.

Critical Property: State is Mutable but Controlled

Unlike a global variable that can be modified arbitrarily, state mutations in LangGraph are mediated by reducer functions.

This ensures that even when multiple nodes modify the state concurrently, the updates are applied consistently and predictably.

Designing the state schema carefully is one of the most consequential decisions when building a LangGraph application.

4.3 Nodes

Nodes are the computational units of a LangGraph graph. Each node is a Python function that encapsulates a specific piece of agent logic. The function signature is standardised: a node receives the current state as its sole argument and returns a dictionary representing the updated state fields.

Examples of what a node might do:

- Invoke an LLM to generate a response or make a decision.

- Call an external tool or API (e.g., a web search, a database query, a code executor).
- Process and transform data (e.g., parse a JSON response, format a prompt).
- Delegate a sub-task to another agent graph.

Node Design Principle

Nodes should be designed to be small, focused, and single-purpose.

A node that does too much violates the separation of concerns and makes the graph harder to debug and modify.

Think of each node as a single, well-defined step in a reasoning chain.

4.4 Edges

Edges define the routing logic of the graph — they determine which node to execute next given the current state. LangGraph supports two types of edges:

Term	Definition
Fixed Edges	An unconditional connection from Node A to Node B. After Node A completes, execution always proceeds to Node B. Used when the next step is always the same regardless of the state.
Conditional Edges	A function-based routing mechanism. The edge function receives the current state and returns the name of the next node to execute. Enables dynamic, state-driven branching — for example, routing to a 'tool_call' node if the LLM requested a tool, or to the 'end' node if the task is complete.

The distinction between fixed and conditional edges is what gives LangGraph its expressive power. Conditional edges enable the agent to dynamically adapt its behaviour based on what has happened so far — making them the primary mechanism for implementing true agentic decision-making.

5. Building Your First Graph: The Five-Step Process

Constructing a LangGraph agent workflow follows a consistent, structured five-step methodology. Each step builds upon the previous, resulting in a fully operational, executable agent graph.

Step 1 — Define the State

The first step is to define the schema of the state object. This involves deciding what information the agent needs to track throughout its execution. The state schema is typically defined as a Python TypedDict, with each field annotated with both its data type and its reducer function.

Common state fields include: the input query, the list of messages exchanged, intermediate tool results, a flag indicating whether the task is complete, and any domain-specific data relevant to the application.

Step 2 — Start the Graph Builder

With the state schema defined, the next step is to initialise the LangGraph StateGraph builder object, passing the state class as its type parameter. The StateGraph builder is the central object that accumulates node and edge definitions before compiling them into an executable graph.

Step 3 — Create and Register Nodes

Each individual step in the agent's reasoning process is implemented as a Python function and registered as a node with the StateGraph builder. The builder is told the name of the node and the function that implements its logic. Nodes are registered one at a time; the order of registration does not determine execution order (that is determined by edges).

Step 4 — Repeat for Additional Nodes

Real-world agent workflows almost always require multiple nodes. Each distinct responsibility — calling an LLM, invoking a tool, processing a result, checking a termination condition — should be encapsulated in its own node. This step is simply repeating Step 3 for as many nodes as the workflow requires.

Step 5 — Define Edges and Compile

The final step connects the nodes by defining edges. At minimum, a graph requires a START edge (connecting the special START node to the first real node) and an END condition (either a fixed edge to the END node, or a conditional edge that can route to END when the task is complete). Once all nodes and edges are registered, the graph is compiled into an executable Runnable by calling the `.compile()` method on the builder.

6. Understanding State and Reducers

State management is the most technically nuanced aspect of LangGraph. A thorough understanding of how state updates are applied — particularly in the presence of concurrent node execution — is essential for building correct and efficient agent workflows.

6.1 The State Update Model

When a node function returns a dictionary of updated state fields, LangGraph does not simply overwrite the existing state with the returned values. Instead, it applies each field update through a corresponding reducer function. A reducer is a function that takes the current value of a field and the new value being proposed, and returns the final merged value.

Term	Definition
Default Reducer (Overwrite)	If no explicit reducer is specified for a field, LangGraph uses a simple overwrite: the new value completely replaces the old value. Suitable for scalar fields like strings, integers, and booleans that represent the latest result.
Append Reducer (operator.add)	The most common explicit reducer. For list fields (such as message histories), the default behaviour is to append new items to the existing list rather than replacing the list. This is achieved using Python's operator.add function as the reducer.
Custom Reducers	Developers can supply any arbitrary function as a reducer. For example, a reducer that takes the maximum of the current and new values, or one that merges two dictionaries by key.

6.2 Why Reducers Enable Parallel Execution

The reducer mechanism is what makes LangGraph's parallel node execution safe. When multiple nodes execute concurrently within the same super-step, they may all attempt to update the same state fields simultaneously. Without reducers, these concurrent updates would conflict and produce non-deterministic results.

By routing all updates through reducer functions, LangGraph can safely collect the outputs of all concurrently executing nodes and merge them into a single, consistent updated state — enabling true parallelism without race conditions or data corruption.

Analogy: Reducers as Thread-Safe State Management

If you are familiar with concurrent programming, reducers serve the same purpose as mutexes or atomic operations.

They ensure that even when multiple threads (nodes) modify shared state (the graph state) simultaneously, the result is always consistent and deterministic.

This is analogous to the reduce step in the MapReduce paradigm used in distributed data processing.

7. Super-Steps and Graph Interactions

The concept of super-steps defines the granularity at which LangGraph executes and manages state during a graph invocation. Understanding super-steps is essential for reasoning about execution order, state persistence, and the boundary of reducer applicability.

7.1 Definition of a Super-Step

A super-step is a single iteration across the currently active nodes of the graph. More precisely:

- Nodes that execute in parallel within the same iteration belong to the same super-step.
- Nodes that execute sequentially — one after another, with the state updated between each — belong to different super-steps.
- Every user-initiated invocation of the graph begins a new, fresh super-step from the START node.

7.2 Reducer Scope: Within vs. Between Super-Steps

A critical and often misunderstood aspect of reducers concerns their scope of applicability:

Reducer Scope Rule

Reducers handle state merging **WITHIN** a super-step: when multiple nodes executing in parallel in the same super-step all update the same field, the reducer merges their outputs into one consistent value.

Reducers do **NOT** apply **BETWEEN** super-steps: after a super-step completes, the final merged state becomes the definitive current state, and the next super-step begins from that definitive state — not from some further reduction.

This distinction is important when designing workflows that involve both parallel and sequential execution.

7.3 Graph Invocation Model

Each call to a compiled LangGraph graph — typically triggered by a user interaction or an external event — initiates a fresh invocation. The invocation provides an initial state (which may be empty or pre-populated with contextual data) and proceeds through the graph's nodes and edges until a terminal condition is reached.

Importantly, state does not automatically persist between separate invocations. If conversational memory is required (e.g., an agent that remembers previous turns of a dialogue), this must be explicitly implemented — either by including conversation history in the state schema, or by using an external persistence mechanism such as LangGraph's checkpointing system.

8. Project Goals and the Sidekick Minimum Viable Product

The theoretical concepts introduced throughout this module are applied in practice through the construction of a production-ready agentic application called 'Sidekick'. The Sidekick MVP demonstrates three fundamental capabilities that are central to building real-world agentic systems.

8.1 Structured Outputs

Structured outputs refer to the practice of constraining LLM responses to conform to a predefined, machine-readable schema (typically a JSON object or a Pydantic model). Rather than receiving an arbitrary free-form text response from an LLM, structured outputs ensure that the response is predictable, parseable, and immediately usable by downstream code.

In the context of multi-agent systems, structured outputs are critically important because:

- Agent A's output is typically Agent B's input. If A's output is unstructured text, B must perform unreliable parsing to extract the information it needs.
- Structured outputs enable type-safe inter-agent communication, dramatically reducing parsing errors and unexpected failures.
- They make agent behaviour more auditable and testable, since the schema acts as a formal contract.

8.2 Multi-Agent Delegation

Multi-agent delegation is the design pattern in which a primary (orchestrator) agent decomposes a complex task into sub-tasks and delegates each sub-task to a specialised (subagent) agent. This pattern enables:

- Specialisation: Each subagent can be optimised for its specific domain (e.g., a coding agent, a web search agent, a data analysis agent).
- Scalability: Complex tasks that would exceed a single LLM's context window or reasoning capacity can be distributed across multiple agents.
- Parallelism: Independent sub-tasks can be delegated to multiple subagents concurrently, reducing total execution time.

In LangGraph, multi-agent delegation is implemented using the graph-of-graphs pattern: a top-level orchestrator graph contains nodes that invoke entire subgraphs (each representing a specialised agent), collect their outputs, and incorporate them into the shared state.

8.3 Playwright Integration

Playwright is Microsoft's open-source, cross-browser automation library. It enables programmatic control of web browsers (Chromium, Firefox, and WebKit), making it a powerful tool for web scraping, browser automation, and end-to-end testing. In the context of the Sidekick MVP, Playwright is used to give the agentic system the ability to interact with web pages — browsing the internet, extracting information, filling forms, and navigating complex web interfaces.

Term	Definition
------	------------

Playwright vs Selenium	Playwright is a modern alternative to Selenium, offering superior performance, more reliable element selection via locators, and native async support. It is the current industry standard for browser automation.
Event Loop Constraint	Playwright currently supports only a single active event loop per process. This creates a challenge in async Python applications that already maintain an event loop (such as LangGraph's async execution engine).
nest-asyncio Solution	The nest-asyncio library (sometimes referred to as 'next-anyio' in informal contexts) resolves the single-event-loop constraint by allowing an existing event loop to be nested — enabling Playwright to be invoked from within another running event loop without conflict.

Production Consideration: Browser Automation at Scale

While Playwright is powerful for single-agent browser interactions, scaling browser automation across multiple concurrent agents introduces significant resource management challenges.

Each browser instance consumes substantial memory. In a production multi-agent system, a browser pool (managing a fixed number of shared browser instances) is typically used to balance capability with resource efficiency.

LangGraph's parallel execution capabilities must be carefully coordinated with Playwright's event loop constraints to avoid race conditions.

SECTION 2 — PRACTICAL & CODING: IPYTHON NOTEBOOK WALKTHROUGH

The following notebook cells constitute a complete, progressive implementation of a LangGraph agent workflow — from environment setup through a fully operational multi-node graph with conditional routing. Each cell is accompanied by a comprehensive line-by-line explanation.

Cell [1] Environment Setup and Imports

Purpose

Install and import all required libraries. This cell establishes the Python environment necessary for building and running a LangGraph graph, including the state schema tools, the graph builder, and the LangChain LLM integration.

```
# Install required packages (run once in your environment)
# pip install langgraph langchain langchain-openai python-dotenv

import os
from typing import TypedDict, Annotated
import operator
from dotenv import load_dotenv

from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, AIMessage, BaseMessage

from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages

# Load API keys from .env file
load_dotenv()
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

print('Environment configured successfully.')
```

Line-by-Line Explanation

from typing import TypedDict, Annotated — TypedDict allows defining state schemas as Python dictionaries with enforced type annotations. Annotated enables attaching metadata (such as reducer functions) to type hints.

import operator — Provides the operator.add function, which serves as the append reducer for list fields such as message histories.

from langgraph.graph import StateGraph, START, END — StateGraph is the core builder class for constructing LangGraph graphs. START and END are special sentinel nodes representing the entry and exit points of every graph.

from langgraph.graph.message import add_messages — A pre-built reducer function provided by LangGraph specifically for message list fields. It handles deduplication and correct ordering of LangChain message objects.

load_dotenv() — Reads environment variables from a .env file. This is the standard pattern for managing API keys securely without hardcoding them in source code.

Cell [2] Define the Agent State Schema

Purpose

Define the state object that will be shared across all nodes in the graph. The state schema determines what information the agent can access and modify throughout its execution. Designing the state carefully is one of the most important architectural decisions when building a LangGraph application.

```
# Define the State TypedDict
# This schema represents everything the agent knows at any point in time

class AgentState(TypedDict):
    # The conversation message history
    # add_messages reducer: appends new messages rather than overwriting
    messages: Annotated[list[BaseMessage], add_messages]

    # The user's original query
    user_query: str

    # Whether the agent has completed its task
    task_complete: bool

    # Accumulated context from tool calls
    context: Annotated[list[str], operator.add]

print('State schema defined:')
print('  - messages:      list[BaseMessage] with add_messages reducer')
print('  - user_query:     str (overwrite reducer)')
print('  - task_complete: bool (overwrite reducer)')
print('  - context:        list[str] with append reducer')
```

Line-by-Line Explanation

class AgentState(TypedDict): — Defines the state schema by inheriting from TypedDict. Every field in this class corresponds to a piece of information that persists and evolves across nodes during a graph invocation.

messages: Annotated[list[BaseMessage], add_messages] — The conversation history field. The Annotated syntax attaches the add_messages reducer, which instructs LangGraph to append new messages to the existing list rather than replacing it. This is the standard pattern for chat-based agents.

user_query: str — A simple string field with no explicit reducer. LangGraph will use the default overwrite reducer — each time a node sets this field, the new value completely replaces the previous one. Suitable for values that represent the current state (not a history).

task_complete: bool — A boolean flag that nodes use to signal that the agent's goal has been achieved. The conditional edge function will read this field to decide whether to continue or terminate.

context: Annotated[list[str], operator.add] — An accumulator for tool call results and external information gathered during the agent's execution. The operator.add reducer means new context items are always appended to the growing list, preserving the full history.

Cell [3] Implement Graph Nodes

Purpose

Define the Python functions that implement the agent's logic. Each function is a node — it receives the current state, performs a specific action, and returns a dictionary of updated state fields. Only the fields present in the returned dictionary are updated; all other fields remain unchanged.

```
# Initialise the LLM
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# — Node 1: Process the user's query —————
def process_query(state: AgentState) -> dict:
    """Invoke the LLM to respond to the user query."""
    messages = state['messages']
    response = llm.invoke(messages)
    return {
        'messages': [response],      # add_messages appends this
        'task_complete': True       # Simple: mark done after one LLM call
    }

# — Node 2: Append context from a simulated tool —————
def fetch_context(state: AgentState) -> dict:
    """Simulate fetching external context (e.g., web search result)."""
    query = state['user_query']
    # In production: call a real tool (Tavily, SerpAPI, etc.)
    simulated_result = f'[Context for: {query}] Key fact retrieved.'
    return {
        'context': [simulated_result] # operator.add appends to list
    }

# — Node 3: Format and return the final answer —————
def format_response(state: AgentState) -> dict:
    """Prepare a structured final response."""
    context_str = '\n'.join(state['context'])
    last_msg = state['messages'][-1].content
    final = f'ANSWER: {last_msg}\nSOURCES: {context_str}'
    return {'messages': [AIMessage(content=final)]}

print('3 nodes defined: process_query, fetch_context, format_response')
```

Line-by-Line Explanation

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0) — Instantiates an OpenAI LLM wrapper. `temperature=0` produces deterministic, reproducible outputs — strongly recommended for agentic workflows where consistency is important.

def process_query(state: AgentState) -> dict: — Node function signature: receives the full current state and returns a dictionary of fields to update. The type annotation `AgentState` improves IDE support and code clarity.

response = llm.invoke(messages) — Sends the conversation history to the LLM and receives an `AIMessage` response. Because `messages` uses the `add_messages` reducer, returning `[response]` appends the AI's response to the history rather than replacing it.

'task_complete': True — The node signals completion by setting this flag. The conditional edge (defined in the next cell) will read this flag to decide whether to end the workflow.

def fetch_context(state: AgentState) -> dict: — A node that simulates a tool call. In a real system, this would invoke a web search API, query a database, or call any external service. It returns only the context field — the messages field remains unchanged.

context_str = '\n'.join(state['context']) — Accesses the accumulated context list from the state and joins all items into a single string for inclusion in the final response. This demonstrates how later nodes can build upon the work of earlier nodes via the shared state.

Cell [4] Build the Graph — Register Nodes and Define Edges

Purpose

Assemble the complete graph by registering the nodes and defining the edges that connect them. This cell also defines the conditional routing logic that gives the agent its decision-making capability. Finally, the graph is compiled into an executable Runnable.

```
# Initialise the StateGraph builder with our state schema
builder = StateGraph(AgentState)

# — Register Nodes —————
builder.add_node('fetch_context', fetch_context)
builder.add_node('process_query', process_query)
builder.add_node('format_response', format_response)

# — Define the Conditional Routing Function —————
def route_after_processing(state: AgentState) -> str:
    """Decide which node to visit after process_query runs."""
    if state.get('task_complete', False):
        return 'format_response' # Task done: go to formatting
    else:
        return 'fetch_context' # Need more context: loop back

# — Define Edges —————
# Fixed edge: START always goes to fetch_context first
builder.add_edge(START, 'fetch_context')

# Fixed edge: After fetching context, always process the query
builder.add_edge('fetch_context', 'process_query')

# Conditional edge: After processing, route based on state
builder.add_conditional_edges(
    'process_query',
    route_after_processing,
    {'format_response': 'format_response', 'fetch_context': 'fetch_context'}
)

# Fixed edge: After formatting, always end
builder.add_edge('format_response', END)

# — Compile the Graph —————
graph = builder.compile()
print('Graph compiled successfully.')
print('Execution path: START -> fetch_context -> process_query')
print('          -> [conditional] -> format_response -> END')
```

Line-by-Line Explanation

builder = StateGraph(AgentState) — Creates the graph builder, parameterised with the AgentState class. All nodes and edges will be registered on this builder before it is compiled. The AgentState type parameter tells LangGraph the schema of the state object.

builder.add_node('fetch_context', fetch_context) — Registers a node with the graph. The first argument is the node's string identifier (used in edge definitions), and the second is the Python function implementing the node's logic.

def route_after_processing(state: AgentState) -> str: — This is the conditional edge function. It takes the current state and returns a string that identifies the next node. The returned string must match one of the keys in the routing dictionary passed to add_conditional_edges.

builder.add_edge(START, 'fetch_context') — Defines a fixed edge from the special START sentinel to the first real node. Every graph must have exactly one START edge.

builder.add_conditional_edges(...) — Registers the conditional routing function. The third argument is a mapping from the function's return values to node names. LangGraph looks up the returned string in this dictionary to determine the actual next node.

graph = builder.compile() — Compiles all registered nodes and edges into an executable Runnable. After compilation, the graph is immutable. Calling compile() validates the graph structure (e.g., checks for unreachable nodes or missing edges) and prepares the execution engine.

Cell [5] Invoke the Graph and Inspect Results

Purpose

Execute the compiled graph with an initial state and examine the outputs. This cell demonstrates the complete invocation pattern, shows how to access the final state after execution, and includes structured output formatting — a core capability of the Sidekick MVP.

```
from pprint import pprint

# — Invoke the Graph —————
initial_state = {
    'messages': [HumanMessage(content='What is LangGraph and how does it work?')],
    'user_query': 'What is LangGraph and how does it work?',
    'task_complete': False,
    'context': []
}

# graph.invoke() runs the graph to completion, returning the final state
final_state = graph.invoke(initial_state)

# — Inspect Final State —————
print('=== FINAL STATE INSPECTION ===')
print(f'Task complete: {final_state["task_complete"]}')
print(f'Messages in history: {len(final_state["messages"])}')
print(f'Context items collected: {len(final_state["context"])}')
print()
print('=== CONVERSATION HISTORY ===')
for msg in final_state['messages']:
    role = 'USER' if isinstance(msg, HumanMessage) else 'AGENT'
    print(f'[{role}]: {msg.content[:120]}...')
```



```
print()

# — Structured Output (Pydantic) —————
from pydantic import BaseModel

class AgentResponse(BaseModel):
    answer: str
    sources: list[str]
    confidence: float

# Bind the Pydantic schema to the LLM for structured outputs
structured_llm = llm.with_structured_output(AgentResponse)
print('Structured output schema bound to LLM successfully')
```

Line-by-Line Explanation

initial_state = {...} — The initial state dictionary provided to the graph. All fields must conform to the AgentState schema. Fields not included here will be absent from the initial state; nodes that access them must handle None or use `.get()` with defaults.

final_state = graph.invoke(initial_state) — The primary invocation method. It runs the graph synchronously from START to END, returning the complete final state dictionary. For asynchronous execution, use `await graph.ainvoke(initial_state)`.

for msg in final_state['messages']: — Iterates over the accumulated message history. Since `messages` uses the `add_messages` reducer, every message added by every node throughout execution is present in this list — providing a complete audit trail of the agent's reasoning.

isinstance(msg, HumanMessage) — Determines the message type to assign the correct role label for display. LangChain's message types (`HumanMessage`, `AIMessage`, `ToolMessage`) enable structured multi-turn conversation management.

class AgentResponse(BaseModel): — Defines a Pydantic model specifying the required structure of the LLM's output. When bound to the LLM using `with_structured_output()`, the LLM is constrained to produce a response that conforms to this schema — guaranteeing parseable, type-safe outputs.

structured_llm = llm.with_structured_output(AgentResponse) — Creates a new LLM wrapper that enforces structured output. Under the hood, LangChain uses either function calling or JSON mode (depending on the LLM provider) to constrain the output. The returned object behaves like a regular LLM but always produces `AgentResponse` objects.

Cell [6] Multi-Agent Delegation Pattern

Purpose

Demonstrate the multi-agent delegation pattern — one of the three core MVP goals. This cell shows how to build a subgraph (specialised agent) and invoke it from within a parent graph node, implementing the orchestrator-subagent architecture in LangGraph.

```
# — Subagent: Web Search Specialist —————
class SearchAgentState(TypedDict):
    query: str
    results: Annotated[list[str], operator.add]
```

```
def web_search_node(state: SearchAgentState) -> dict:
    """Simulates a web search tool call."""
    # In production: use Tavily, SerpAPI, or Playwright
    result = f'[SEARCH RESULT for "{state["query"]}"] LangGraph is a...'
    return {'results': [result]}

# Build and compile the subagent graph
search_builder = StateGraph(SearchAgentState)
search_builder.add_node('web_search', web_search_node)
search_builder.add_edge(START, 'web_search')
search_builder.add_edge('web_search', END)
search_agent = search_builder.compile()

# — Orchestrator Node: Delegates to the Subagent —————
def orchestrator_node(state: AgentState) -> dict:
    """Orchestrator: delegates web search to the search subagent."""
    # Invoke the subagent as a function call
    search_result = search_agent.invoke({
        'query': state['user_query'],
        'results': []
    })
    # Extract results and merge into the parent state
    gathered_context = search_result['results']
    return {'context': gathered_context}

# Test the delegation pattern
test_state = {'user_query': 'LangGraph tutorial', 'context': []}
orchestrator_output = orchestrator_node(test_state)
print('Delegation result:', orchestrator_output)
```

Line-by-Line Explanation

class SearchAgentState(TypedDict): — The subagent has its own independent state schema, entirely separate from the parent orchestrator's state. This isolation is a key design principle: subagents encapsulate their own state, preventing leakage or interference with the parent graph's state.

search_agent = search_builder.compile() — The subagent is itself a compiled LangGraph graph. This is the graph-of-graphs pattern: the orchestrator graph contains nodes that invoke entire subgraphs. Each subgraph can have its own nodes, edges, state, and routing logic.

def orchestrator_node(state: AgentState) -> dict: — A node in the parent orchestrator graph that acts as a delegation point. It takes the parent state, extracts the relevant information, invokes the subagent graph, and merges the subagent's results back into the parent state.

search_result = search_agent.invoke({...}) — Invokes the subagent graph synchronously. The subagent runs to completion and returns its final state. The orchestrator node then extracts the relevant results from the subagent's state.

return {'context': gathered_context} — Only the fields being updated are included in the return dictionary. The operator.add reducer on the parent's context field will append the subagent's results to any previously accumulated context — preserving all prior work.

CHAPTER SUMMARY — LANGGRAPH WEEK 4

Term	Definition
LangChain Ecosystem	LangChain (core framework) + LangGraph (orchestration) + LangSmith (monitoring) form a complete Agentic AI development platform.
LangGraph Purpose	Low-level, stateful, graph-based orchestration framework for building controllable, cyclical multi-agent workflows.
LangGraph Studio	Visual builder for designing and debugging agent workflows interactively.
LangGraph Platform	Production infrastructure for deploying and scaling LangGraph agents with REST APIs and state persistence.
Graphs	Agent workflows represented as directed graphs with nodes (logic) and edges (routing). Supports cycles, branches, and parallelism.
State	A shared, mutable TypedDict object representing the agent's complete information at any point during execution.
Nodes	Python functions: receive current state, perform an action (LLM call, tool invocation, etc.), return updated state fields.
Edges	Fixed (unconditional) or Conditional (function-based routing). Conditional edges are the primary mechanism for agentic decision-making.
Reducers	Functions that merge new field values with existing state. Enable safe concurrent node execution. Default: overwrite. Common: add_messages, operator.add.
Super-Steps	A single iteration across active nodes. Parallel nodes share a super-step; sequential nodes belong to different super-steps. Reducers apply within, not between, super-steps.
Structured Outputs	Pydantic-schema-constrained LLM outputs ensuring predictable, type-safe inter-agent communication via llm.with_structured_output().
Multi-Agent Delegation	Orchestrator graph invokes compiled subgraphs as function calls — the graph-of-graphs pattern for decomposing complex tasks.
Playwright Integration	Browser automation for web-interacting agents. Requires nest-asyncio to resolve single-event-loop constraints when used within LangGraph's async runtime.

End of Week 4: LangGraph — Agentic AI

This document has covered the complete Week 4 curriculum on LangGraph — from the foundational LangChain ecosystem through the theoretical underpinnings of graph-based agent orchestration, to a comprehensive practical notebook demonstrating state management, node and edge construction, conditional routing, structured outputs, multi-agent delegation, and Playwright integration. Each concept has been presented with both rigorous theoretical grounding and annotated, production-oriented code examples.

